

Chapitre 2 : Contrôles

TD / TP

Table des matières

1	Travaux Dirigés	2
1.1	Expressions	2
1.2	Analyse de programmes	2
1.2.1	Programme	2
1.2.2	Programme	2
1.2.3	Programme	3
1.2.4	Programme	3
1.2.5	Programme	3
1.3	Ecriture de programmes	3
1.3.1	Inverse.....	3
1.3.2	Multiplications	3
1.3.3	Triangle.....	3
1.3.4	Contient X.....	3
2	Travaux Pratiques	4
2.1	Consignes importantes	4
2.2	Exercices	4
2.2.1	min_max.py	4
2.2.2	occurrences_caractere.py	5
2.2.3	repetition.py	5
2.2.4	indice_caractere.py	5
2.2.5	extraire_N_M.py	5
2.2.6	nombre_voyelles.py	5
2.2.7	est_palindrome.py	5
2.2.8	est_palindrome_etendu.py	6
2.2.9	indice_chaine.py	6
2.2.10	mots_phrase.py	6
2.3	Remise	6

1 Travaux Dirigés

Vous réalisez les exercices suivants sur papier, sans utiliser l'interpréteur Python.

1.1 Expressions

Indiquez le type et la valeur des expressions suivantes.

Expression	Type	Valeur	Commentaire
len((True, 'hi !', 3.14))			
len(True, 'hi !', 3.14)			
len("")			
'abcdefg'[4]			
'abcdefg'[7]			
'abcdefg'[-3]			
'abcdefg'[-7]			
'abcdefg'[-8]			
'abcdefg'[1:4]			
'abcdefg'[4:5]			
'abcdefg'[3:3]			
'abcdefg'[3:2]			
'abcdefg'[:2]			
'abcdefg'[4:]			
'abcdefg'[:]			
(True, 'hi !', 3.14)[1]			
(True, 'hi !', 3.14)[0:2]			
(True, 'hi !', 3.14)[-1]			
'abcdefg'[1:7:3]			
'abcdefg'[-2:-6:-2]			
5 if False else 6			

1.2 Analyse de programmes

Indiquez si les programmes suivants sont corrects ou pas. Si le programme est incorrect, corrigez-le. S'il est correct, expliquez ce qu'il fait et affiche, et le cas échéant, améliorez-le.

1.2.1 Programme

```
n = int(input('Entrer un entier : '))
i = 1
somme = 0
while i <= n:
    somme += i
    i += 1
print('somme =', somme)
```

1.2.2 Programme

```
for i in range(5,0):
    print(i)
```

1.2.3 Programme

```
for i in range(10):  
    if i <= 3:  
        continue  
    if i > 8:  
        break  
    print(i)
```

1.2.4 Programme

```
s = 'Bonjour à tous !'  
i = 1  
while i < 9:  
    print(s[i], end='')  
    i += 1
```

1.2.5 Programme

```
n = int(input('entier ? '))  
print(n, 'est', 'pair' if n % 2 == 0 else 'impair')
```

1.3 Ecriture de programmes

Ecrivez *sur papier* les programmes de quelques lignes suivants.

1.3.1 Inverse

Demande à l'utilisateur une chaîne de caractères et affiche cette chaîne de façon inversée (le premier caractère est le dernier, et vice-versa).

1.3.2 Multiplications

Affiche la table de multiplication des nombres compris entre 0 et 10 inclus.

1.3.3 Triangle

Demande à l'utilisateur un entier et affiche un triangle composé de 'X' ayant la hauteur correspondante.

1.3.4 Contient X

Demande à l'utilisateur une chaîne de caractères quelconque et affiche True si la chaîne contient la lettre 'X' (en majuscule) et False sinon.

Contrainte : le script utilise uniquement une boucle for sur séquence, l'opérateur booléen `in` (non étudié en cours) est interdit.

2 Travaux Pratiques

2.1 Consignes importantes

Pour vous familiariser dès maintenant aux conditions d'examen, réalisez les exercices suivants dans l'environnement de développement existant sur les machines EPITA, c'est-à-dire avec votre machine virtuelle Linux, un simple éditeur de texte et l'interpréteur Python.

Après avoir ouvert un terminal Linux, créez un répertoire dans lequel vous réaliserez les exercices de ce TP (le nom est bien sûr indicatif) :

```
$ mkdir -p S1-IPYT/TP2  
$ cd S1-IPYT/TP2
```

Pour une meilleure productivité, il est conseillé d'ouvrir deux terminaux : dans le premier vous écrivez votre script avec un éditeur, dans le second vous le testez avec l'interpréteur Python. Vous gardez l'éditeur ouvert tant que votre script ne fonctionne pas correctement et sauvegardez sur disque chaque nouvelle version avant de la tester.

Les exercices sont faits pour que vous appliquiez les constructions et fonctions que nous avons étudiées en cours. Vous **ne devez pas** en utiliser d'autres, notamment les fonctions de manipulation de chaînes de caractères. Certains exercices sont assortis de contraintes de réalisation supplémentaires que vous devez respecter.

2.2 Exercices

Les paragraphes suivants décrivent les exercices à réaliser, classés par ordre croissant de difficulté. Chaque exercice consiste à développer un script Python, dont le nom est le titre de l'exercice. Le paragraphe qui suit spécifie ce que le script doit faire et donne des exemples.

Vous devez tester votre script dans le plus de cas possible – cas standard, cas particuliers et cas d'erreur – en faisant varier les entrées utilisateur. Le testeur automatique qui sera fourni ne fait que deux tests par exercice : un test correspondant à un cas normal d'utilisation, un autre correspondant à un cas particulier ou à un cas d'erreur. Le testeur qui établira votre note finale fera des tests plus complets et vérifiera si les contraintes de réalisation sont respectées.

Dans la suite du document : (i) le terme « caractère » s'entend comme « chaîne de caractères de longueur 1 », (ii) pour plus de clarté, les caractères sont délimités par des guillemets simples, les chaînes par des guillemets doubles, comme en C.

2.2.1 min_max.py

Demande à l'utilisateur une chaîne de caractères quelconque et affiche sur une ligne, séparés par un espace, le plus petit et le plus grand caractère de la chaîne, au sens de l'opérateur standard <. Si la chaîne saisie est vide, le script affiche "vide". Exemple : pour la chaîne "bonjour", le script affiche "b u".

Contrainte : le script n'utilise que les boucles for sur séquence et l'indigage.

2.2.2 occurrences_caractere.py

Demande à l'utilisateur une chaîne de caractères quelconque, puis un caractère quelconque, et affiche le nombre d'occurrences du caractère dans la chaîne. Le script affiche "erreur" si la seconde entrée utilisateur n'est pas un caractère. Exemple : dans la chaîne "bonjour" le nombre d'occurrence du caractère 'o' est 2, celui de 'x' est 0.

Contrainte : le script n'utilise que l'indigage et la comparaison de caractères.

2.2.3 repetition.py

Demande à l'utilisateur une chaîne de caractères quelconque et affiche True si la chaîne contient un caractère qui est répété, avec la même casse, et False sinon. Exemple : la chaîne "bonjour" contient une répétition, la chaîne "bOnjour" n'en contient pas.

Contrainte : le script n'utilise que l'indigage et la comparaison de caractères.

2.2.4 indice_caractere.py

Demande à l'utilisateur une chaîne de caractères quelconque, puis un caractère quelconque, et affiche l'indice de la première occurrence du caractère dans la chaîne, s'il est présent, ou -1 sinon. Le script affiche "erreur" si la seconde entrée utilisateur n'est pas un caractère. Exemple : dans la chaîne "bonjour", l'indice du caractère 'o' est 1, celui de 'x' est -1.

Contrainte : le script n'utilise que l'indigage et la comparaison de caractères.

2.2.5 extraire_N_M.py

Demande à l'utilisateur une chaîne de caractères quelconque, un entier N positif ou nul, un entier M strictement supérieur à N, et affiche sur une ligne, sans séparateur et à l'envers, tous les caractères de la chaîne dont l'indice est congru à N modulo M. Exemple : pour la chaîne "bonjour", N = 2 et M = 3, le script affiche "un".

Remarque : il n'est pas demandé de tester la validité des entiers N et M saisis.

2.2.6 nombre_voyelles.py

Demande à l'utilisateur une chaîne de caractères quelconque et affiche le nombre de voyelles minuscules ou majuscules que la chaîne contient. Exemple : dans la chaîne "BonjOur", le nombre de voyelles est 3.

Contrainte : le script n'utilise que des boucles for sur séquence et n'utilise pas l'opérateur booléen `in`¹.

2.2.7 est_palindrome.py

Demande à l'utilisateur une chaîne de caractères quelconque et affiche True si la chaîne est un palindrome et False sinon. Une chaîne est un palindrome ssi elle se lit à l'identique de gauche à droite et de droite à gauche, casse incluse. Exemples : "bob", "ELLE" et "kayak" sont des palindromes, et "Bob" n'en est pas un. Par convention, la chaîne vide est un palindrome.

¹ non encore étudié en cours

Contraintes : le script n'utilise que l'indigage et la comparaison de caractères.

2.2.8 est_palindrome_etendu.py

Même exercice que `est_palindrome.py`, avec les mêmes contraintes et les fonctionnalités complémentaires suivantes : (1) la comparaison est insensible à la casse, de sorte que "Bob" est désormais un palindrome, (2) les espaces sont ignorés, de sorte que "Ta bete te bat"² est désormais un palindrome.

Remarque : les lettres accentuées restent non traitées : « é », « è », « ê » et « e » sont considérées comme différentes, et "Ta bête te bat" n'est pas un palindrome.

2.2.9 indice_chaine.py

Demande à l'utilisateur deux chaînes de caractères, `chaine1` et `chaine2`, et affiche l'indice de la première occurrence de `chaine2` dans `chaine1`, si elle existe, ou -1 sinon. Exemple : dans la chaîne "bonjour", l'indice de la chaîne "jour" est 3, celui de la chaîne "oj" est -1.

Contraintes : le script n'utilise que l'indigage et la comparaison de caractères.

2.2.10 mots_phrase.py

Demande à l'utilisateur une chaîne de caractères quelconque et affiche sur une même ligne, séparés par une virgule, les différents mots qui la compose. Un mot est une suite de caractères contigus quelconques délimitée par le début de la chaîne, la fin de la chaîne ou un ou plusieurs espaces ou tabulations. Exemple : pour la chaîne "Bonjour ! comment ça va ?", le script affiche "Bonjour,!,comment,ca,va,?".

Contraintes : le script n'utilise que l'indigage et la comparaison de caractères.

2.3 Remise

Remettez svp vos fichiers .py dans l'espace Moodle prévu à cet effet.

Vous devez les remettre à *plat*, c'est-à-dire sans répertoires ou sous-répertoires intermédiaires et sans les regrouper en archive, compressée ou non. En résumé, faites un simple glisser-déposer des différents fichiers depuis votre répertoire de développement dans l'espace Moodle.

Vous pouvez remettre un ou plusieurs de vos fichiers autant de fois que vous le voulez jusqu'à la date limite de remise. Toute nouvelle version doit *écraser* la précédente : les fichiers nommés par exemple `script ( 1 ) .py` seront ignorés lors de la correction.

Notez que l'espace Moodle est configuré pour accepter les remises hors délai, mais alors les pénalités de retard s'appliquent.

² Louise de Villemorin